Computer Organisation

Topic 8
Control Unit

Stallings, Computer Organization & Architecture (8th Ed.), Chap. 15 & 16

Hamacher et al., Computer Organisation (5th Ed.), Chap. 7

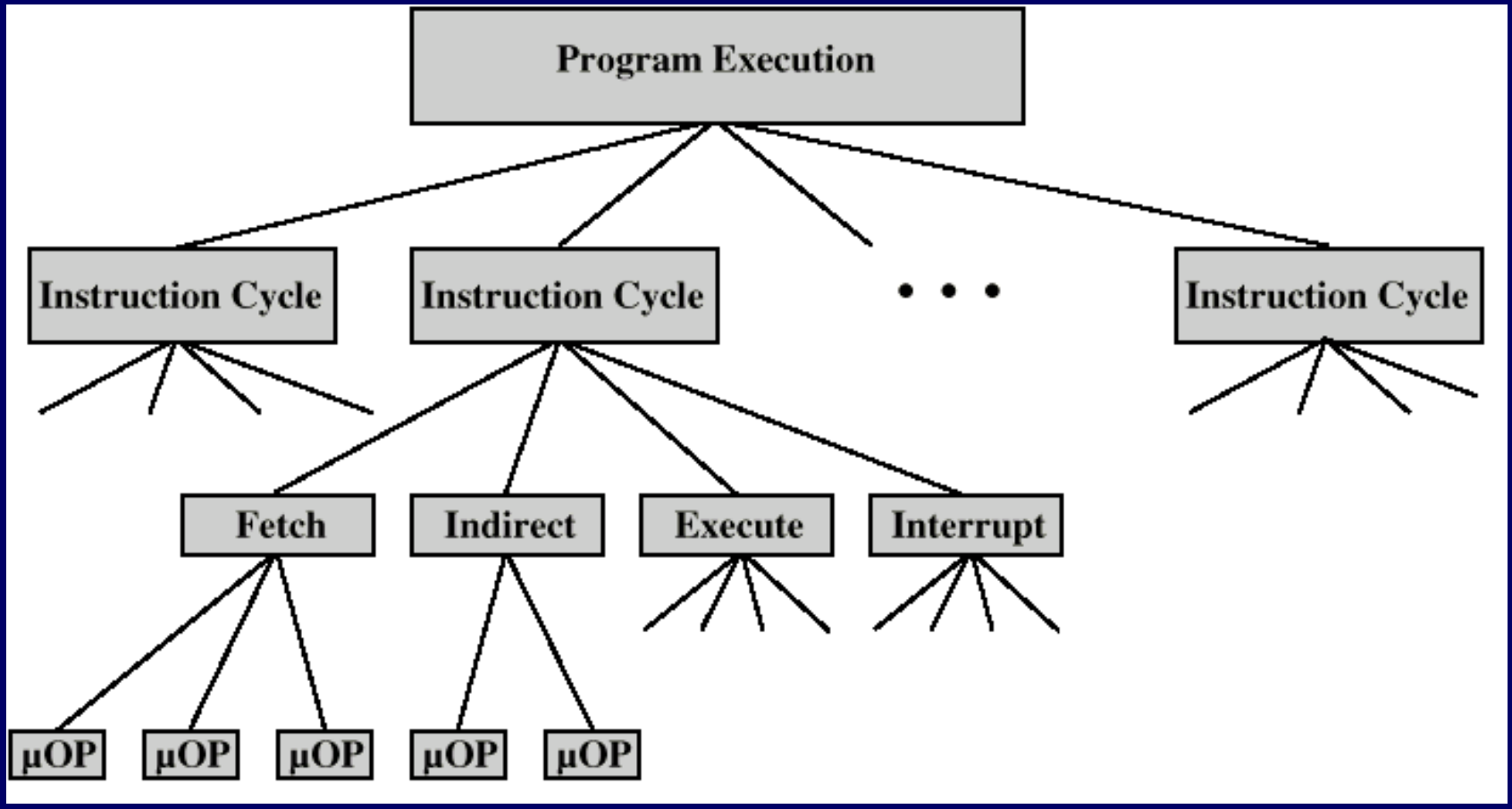
Outline

- Control Unit
  - Micro-operations
    - Fetch, Indirect, Interrupt and Execute cycles
    - Instruction cycle
  - Control of the Processor
    - Functional requirements and control signals
- Hardwired Implementation
- Micro-programmed Control
  - Basic concepts
    - Horizontal/Vertical microprogramming and control memory
  - Microinstruction Sequencing
    - Design and sequencing
    - Addressing
  - Microinstruction Execution
    - Taxonomy and encoding

Micro-Operations

- A computer executes a program
- A program executes instructions sequentially (executed in an instruction cycle)
- Each instruction cycle made up of shorter subcycles
  - e.g. fetch, indirect, execute, interrupt
  - see pipelining
- Each subcycle involves shorter operations called micro-operations (each μ-op does very little, atomic operation of CPU)

Constituent Elements of Program Execution



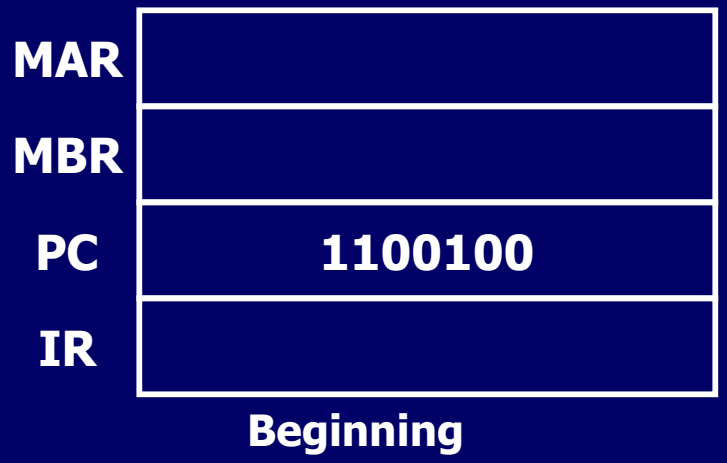
Fetch - 4 Registers

- Memory Address Register (MAR)
  - Connected to address bus
  - Specifies address for read or write op
- Memory Buffer Register (MBR)
  - Connected to data bus
  - Holds data to write or last data read
- Program Counter (PC)
  - Holds address of next instruction to be fetched
- Instruction Register (IR)
  - Holds last instruction fetched

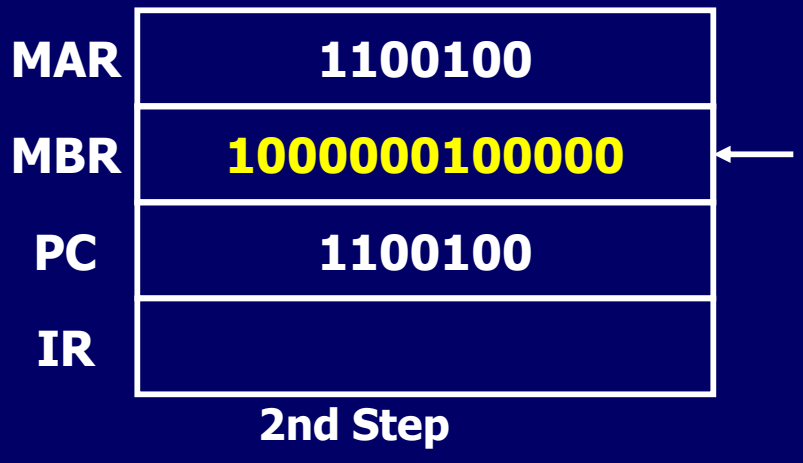
Fetch Sequence (1)

- Address of next instruction is in PC
- 1st Step: Move address from PC to MAR (because MAR is connected to address bus)
- 2nd Step: Bring in instruction
  - address in MAR is placed on address bus
  - control unit issues READ command on control bus
  - result (data from memory) appears on data bus
  - data from data bus copied into MBR
  - PC++ (in parallel with data fetch from memory)
- 3rd Step: Data (instruction) moved from MBR to IR
- MBR is now free for further data fetches

Fetch Sequence (2)

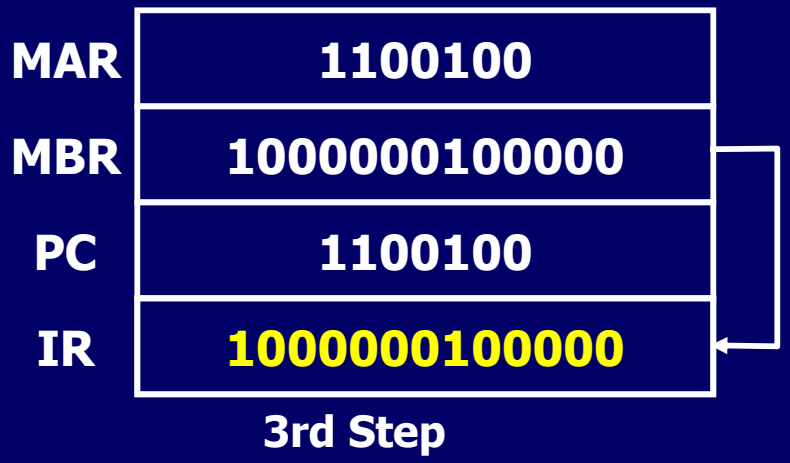


Beginning



2nd Step

1st Step



3rd Step

Fetch Sequence (symbolic) - 4 Micro-ops

- t1: MAR <- (PC)
- t2: MBR <- Memory
- PC <- (PC) +I
- t3: IR <- (MBR)
  - (tx = time unit/clock cycle)
  - I = instruction length

or

- t1: MAR <- (PC)
- t2: MBR <- Memory
- t3: PC <- (PC) +I
- IR <- (MBR)

Rules for Clock Cycle Grouping

- Proper sequence must be followed
  - MAR <- (PC) must precede MBR <- (memory)
- Conflicts must be avoided
  - Must not read & write same register at same time
  - MBR <- (memory) & IR <- (MBR) must not be in same cycle
- Also: PC <- (PC) +I involves addition
  - Use ALU
  - May need additional micro-operations

Indirect Cycle

- For indirect address of operands
- MAR <- ($IR_{address}$) - address field of IR
- MBR <- (memory)
- $IR_{address}$ <- ($MBR_{address}$)

- MBR contains an address
- IR is now in same state as if direct addressing had been used

Interrupt Cycle

- At the end of execute cycle, test if interrupts have occurred.
- t1: MBR <- (PC)
- t2: MAR <- save-address
- PC <- routine-address
- t3: memory <- (MBR)
- This is a minimum
  - May be additional micro-ops to get addresses
  - N.B. saving context is done by interrupt handler routine, not micro-ops

saved for return from the interrupt

location where content of PC is to be saved

address of interrupt routine

store old PC value to memory

Execute Cycle (ADD)

- Different for each instruction
- e.g. ADD R1, X - add the contents of location X to register R1 , result in R1
- t1: MAR <- ($IR_{address}$)
- t2: MBR <- Memory
- t3: R1 <- (R1) + (MBR)

IR has ADD instruction. Address portion of IR is loaded into MAR (operand)

referenced memory is read

- The above is simplified. Additional micro-ops may be required

Execute Cycle (ISZ)

- ISZ X - increment and skip if zero
  - t1: MAR <- ($IR_{address}$)
  - t2: MBR <- Memory
  - t3: MBR <- (MBR) + 1
  - t4: Memory <- (MBR)
  - IF ((MBR) = 0) THEN (PC <- (PC) + I)
- Notes:
  - IF is a single micro-operation
  - Micro-operations done during t4

IR has ISZ instruction. Address portion of IR is loaded into MAR

referenced memory is read

Execute Cycle (BSA)

- BSA X - Branch and save address
  - Address of instruction following BSA is saved in X
  - Execution continues from X+I
  - t1: MAR <- ($IR_{address}$)
  - MBR <- (PC)
  - t2: PC <- ($IR_{address}$)
  - memory <- (MBR)
  - t3: PC <- (PC) + I

IR has BSA instruction. Address portion of IR is loaded into MAR (to store)

saved for return from the branch

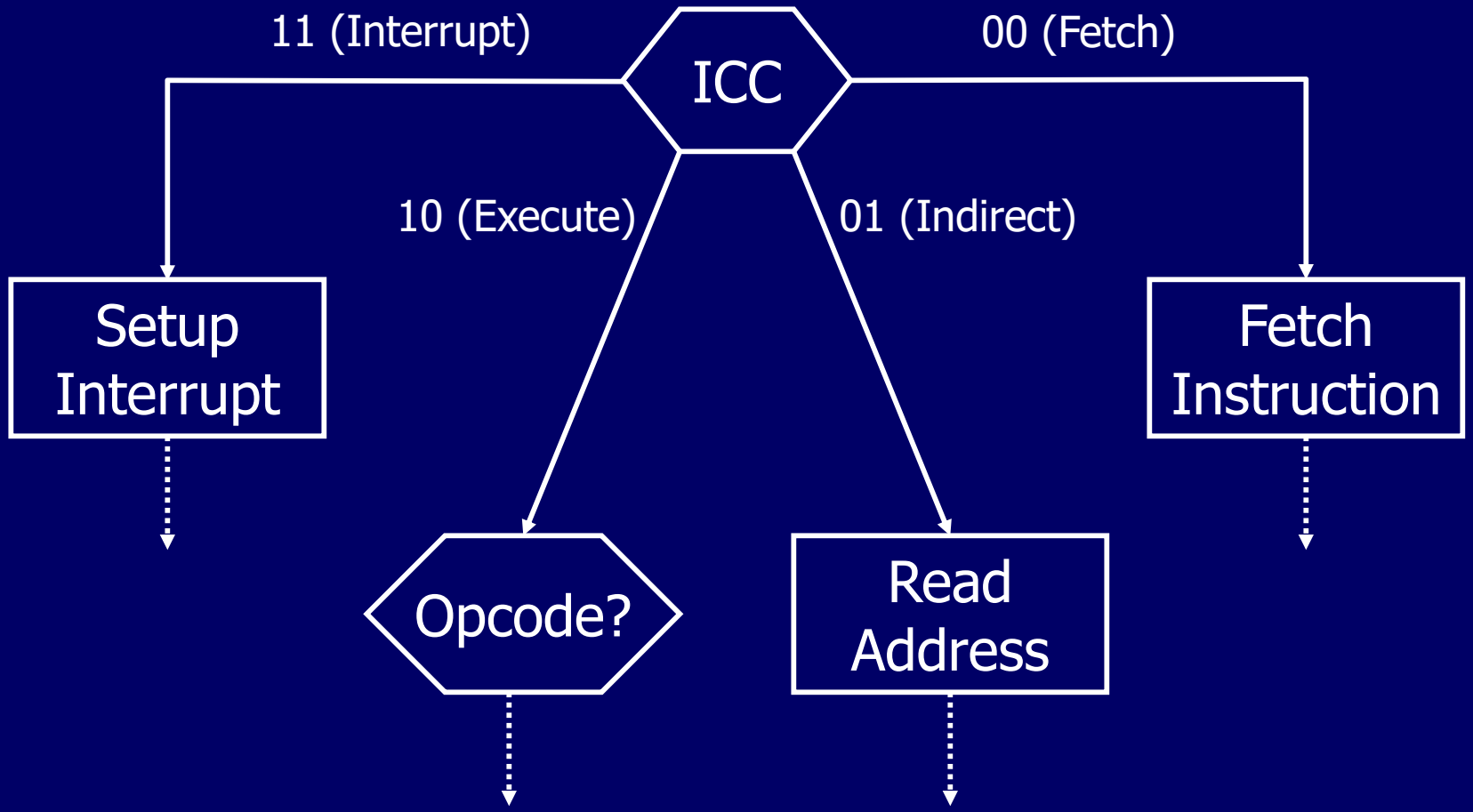
address of branch routine

store old PC value to memory

Instruction Cycle Code

- There is a need to tie sequences of micro-ops together (with 2-bit register = ICC)
- ICC designates the state of the processor:
  - 00: Fetch
  - 01: Indirect
  - 10: Execute
  - 11: Interrupt
- See Figure 15.3 in Stallings p 587

Instruction Cycle Flowchart



Outline

- Control Unit
  - Micro-operations
    - Fetch, Indirect, Interrupt and Execute cycles
    - Instruction cycle
  - Control of the Processor
    - Functional requirements and control signals
- Hardwired Implementation
- Micro-programmed Control
  - Basic concepts
    - Horizontal/Vertical microprogramming and control memory
  - Microinstruction Sequencing
    - Design and sequencing
    - Addressing
  - Microinstruction Execution
    - Taxonomy and encoding

Control Unit Functional Requirements

- i.e. functions that the control unit must perform
- To characterise the control unit we need to:
  - Define basic elements of processor
  - Describe micro-operations processor performs
  - Determine functions control unit must perform

Basic Elements of Processor

- ALU: functional centre of the computer
- Registers: processor data storage
- Internal data paths: to move data between registers and between register and ALU
- External data paths: link registers to memory and IO modules (system bus)
- Control Unit: causes operations to happen within the processor
- Program execution has operations involving these processor elements

Types of Micro-operation

- Operations consist of a sequence of micro-ops
- Categories of micro-ops:
  - Transfer data between registers
  - Transfer data from register to external interface (system bus)
  - Transfer data from external interface to register
  - Perform arithmetic or logical ops

Functions of Control Unit

- Control unit performs 2 basic tasks:
  - Sequencing
  - Execution
- Sequencing
  - Causing the CPU to step through a series of micro-operations in correct sequence
- Execution
  - Causing the performance of each micro-op
- This is done using Control Signals
  - i.e. control unit produces control signals

Control Signals - Inputs (1)

- For Control Unit to perform, it must have:
  - inputs to determine the state of the system
  - outputs to control the behaviour of the system
- Clock
  - One micro-instruction (or set of parallel micro-instructions) per clock cycle
- Instruction register
  - Op-code for current instruction
  - Determines which micro-instructions are performed

Control Signals - Inputs (2)

- Flags
  - To determine status of CPU
  - Results of previous ALU operations
  - e.g. ISZ instruction
- Control signals from control bus
  - Interrupts
  - Acknowledgements

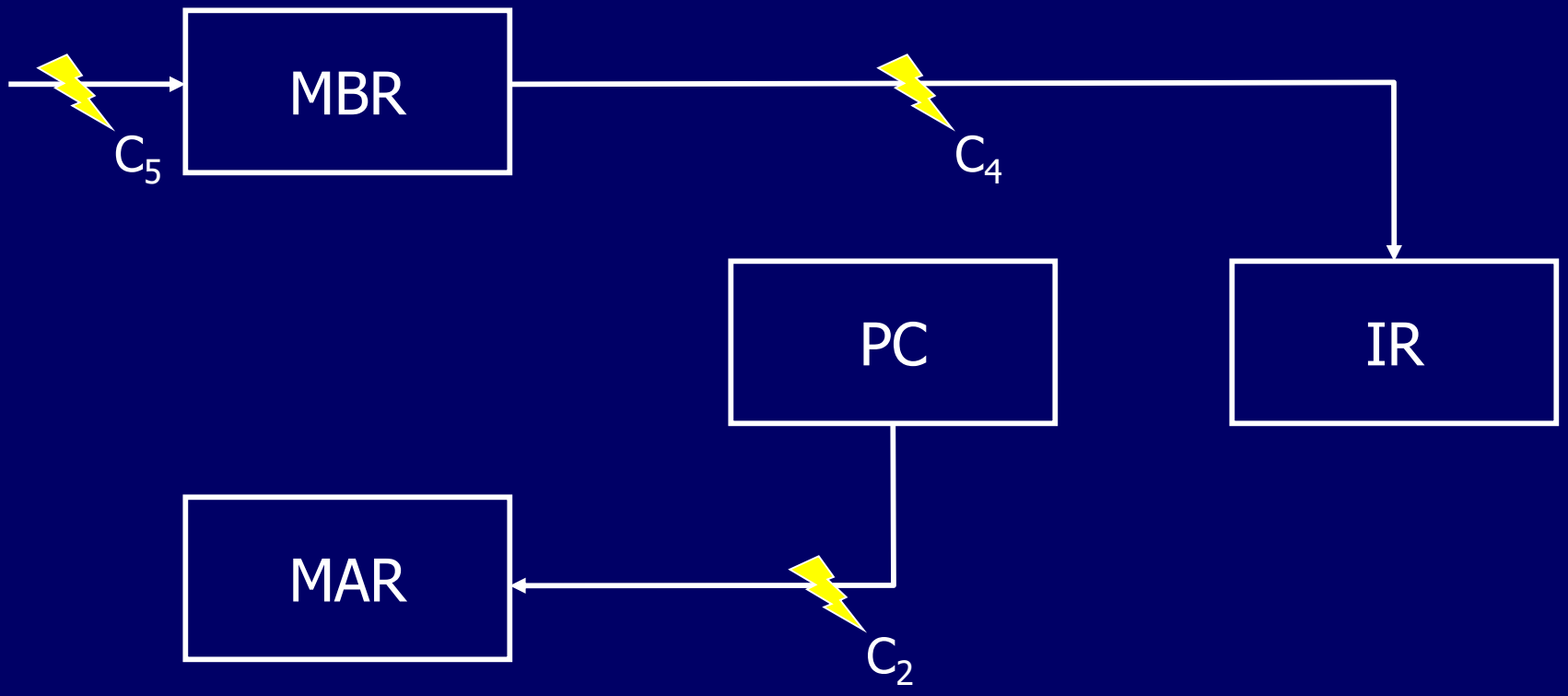
Control Signals - Output

- Control signals within CPU (2 types)
  - Cause data movement (between registers)
  - Activate specific ALU functions
- Control signals to control bus (2 types)
  - To memory
  - To IO modules

Example Control Signal Sequence - Fetch

- MAR <- (PC)
  - Control unit activates signal to open gates between PC and MAR
- MBR <- Memory, PC <- (PC) + 1
  - Open gates between MAR and address bus
  - Memory read control signal
  - Open gates between data bus and MBR
  - Signal to ALU to add 1 to contents of PC
- IR <- (MBR)
  - Open gates between MBR and IR
- Decide for indirect cycle (check IR if indirect)

Data Paths and Control Signals e.g. Fetch



See Figure 15.5 & Table 15.1 in Stallings pp 590/591

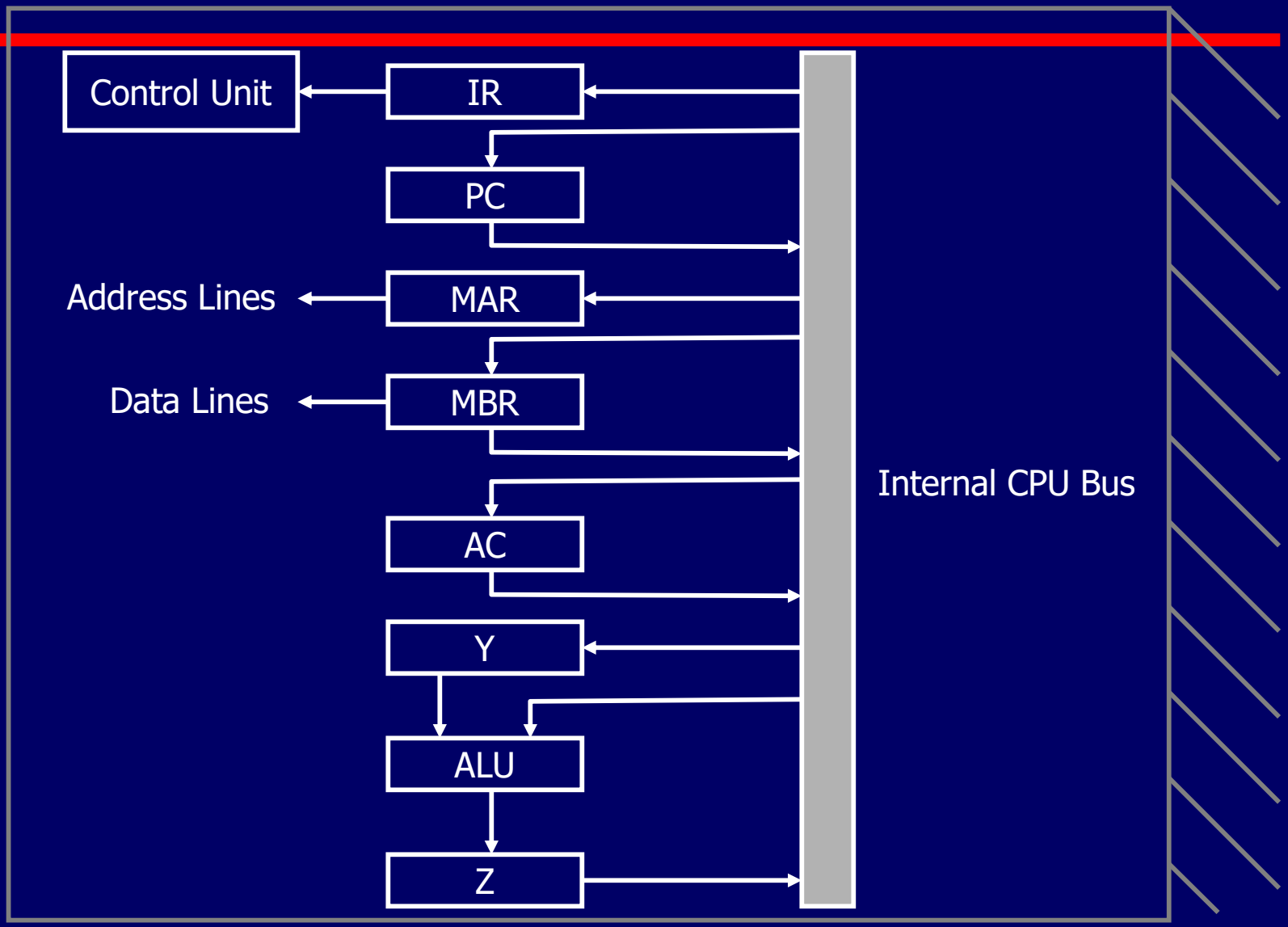
Data Paths and Control Signals

- With each clock cycle, control unit reads all inputs and emits a set of control signals
- Control signals go to 3 separate destinations:
  - Data paths: Control unit controls internal flow of data. Control signal from Control unit opens gate to let data pass
  - ALU: Control unit controls the operation of the ALU (by activating various logic devices and gates on ALU)
  - System bus: Control unit sends control signals out onto the control lines of the system bus (e.g. memory READ)

Internal Organization (1)

- ALU and all processor registers usually connected by a single internal bus
- Gates and control signals are used to control data movement onto and off the bus from each register
- Additional control signals control data transfer to and from external systems bus and ALU operations

CPU with Internal Bus



Internal Organization (2)

- Temporary registers (Y & Z) needed for proper operation of ALU (temporary input/output storage)
  - t1: MAR <- ($IR_{address}$)
  - t2: MBR <- Memory
  - t3: Y <- (MBR)
  - t4: Z <- (AC) + (Y)
  - t5: AC <- (Z)
- Other organization possible but internal bus is a popular feature

IR has instruction. Address portion of IR is loaded into MAR

referenced memory is read

Outline

- Control Unit
  - Micro-operations
    - Fetch, Indirect, Interrupt and Execute cycles
    - Instruction cycle
  - Control of the Processor
    - Functional requirements and control signals
- Hardwired Implementation
- Micro-programmed Control
  - Basic concepts
    - Horizontal/Vertical microprogramming and control memory
  - Microinstruction Sequencing
    - Design and sequencing
    - Addressing
  - Microinstruction Execution
    - Taxonomy and encoding

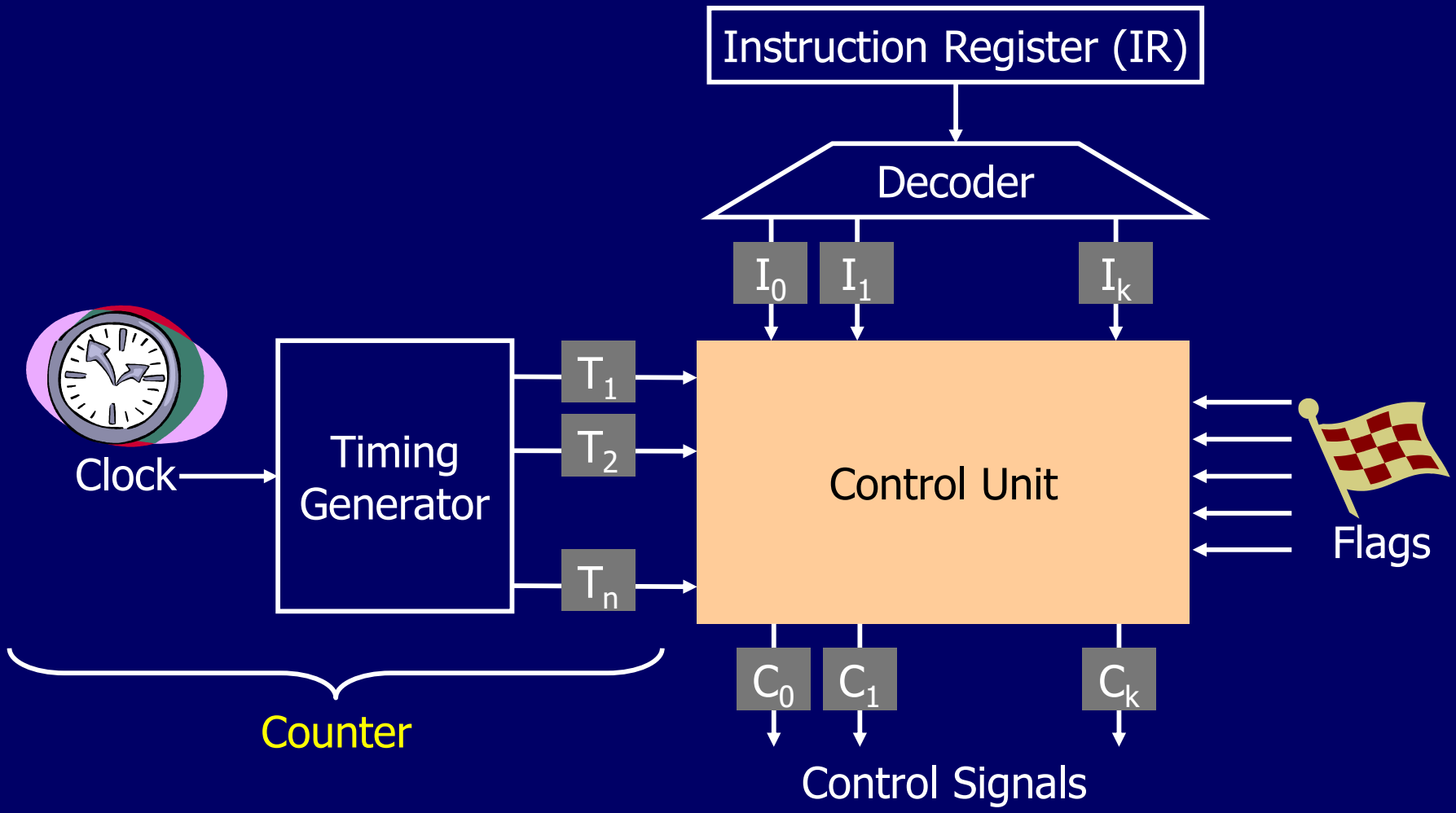
Hardwired Implementation Control Unit Inputs (1)

- Control unit inputs (instruction register, clock, flags & control bus signals)
- Flags and control bus
  - Each bit means something (e.g. overflow)
- Instruction register
  - Op-code causes different control signals for each different instruction
  - To simplify control unit logic, there should be unique logic for each op-code (use decoder)
  - Decoder takes encoded input and produces single output (decoder has n inputs bits and 2^n outputs)

Hardwired Implementation Control Unit Inputs (2)

- Clock
  - Repetitive sequence of pulses
  - Useful for measuring duration of micro-ops
  - Must be long enough to allow signal propagation
  - Different control signals at different times within instruction cycle
  - Need a counter with different control signals for t1, t2 etc.
    - counter reset to t1 at end of instruction cycle

Hardwired Implementation Control Unit Inputs (3)

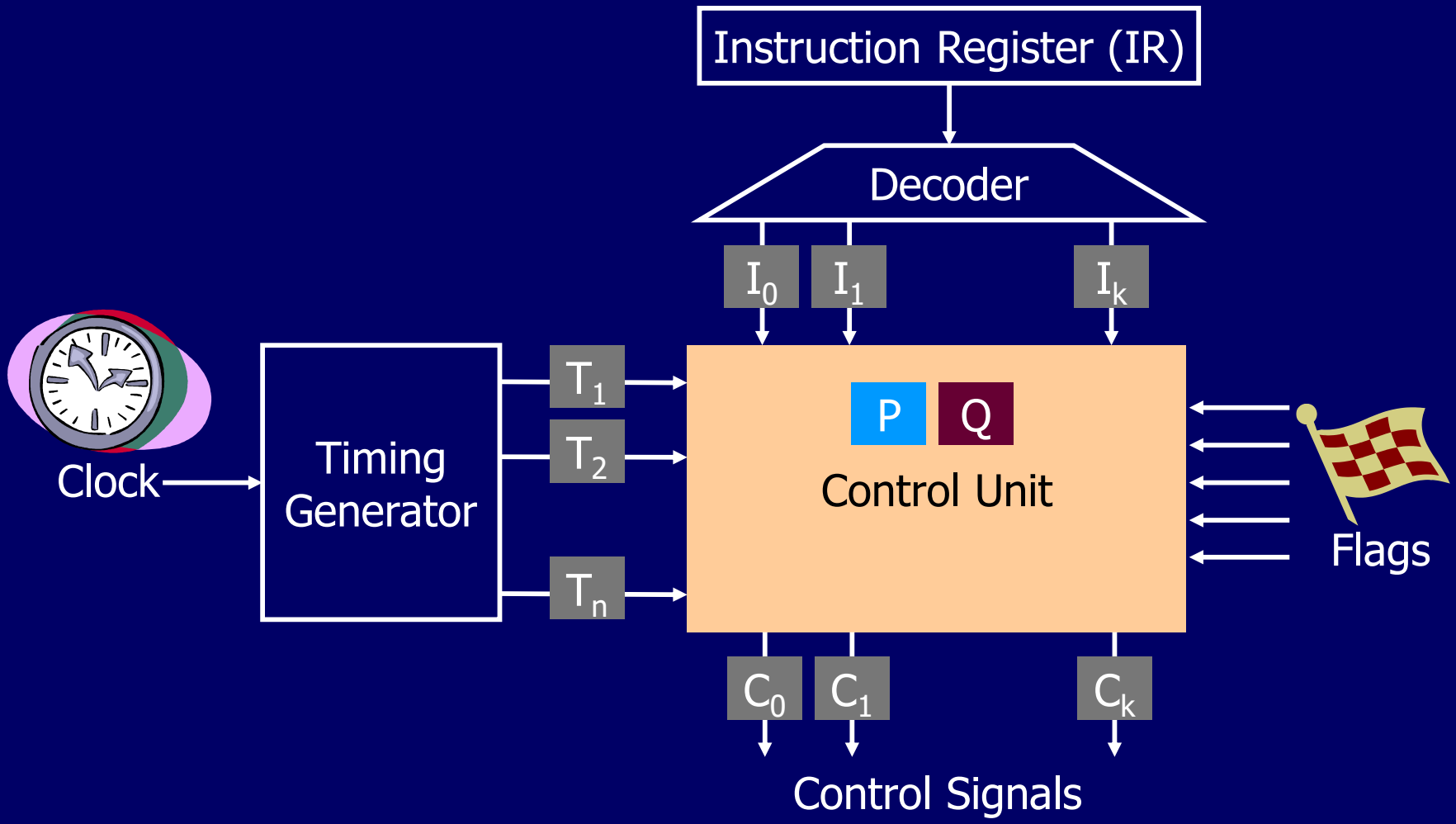


Hardwired Implementation Control Unit Logic (1)

- For each control signal, need to derive a Boolean expression of the signal as a function of the inputs
- e.g. P & Q are control signals:
  - PQ = 00 Fetch Cycle
  - PQ = 01 Indirect Cycle
  - PQ = 10 Execute Cycle
  - PQ = 11 Interrupt Cycle
  - For control signal, e.g. C5 for Fetch and Indirect at T_2, the expression is $C5 = \overline{P} \bullet \overline{Q} \bullet T_2 + \overline{P} \bullet Q \bullet T_2$

MBR <- Memory

Hardwired Implementation Control Unit Logic (2)



Problems With Hardwired Designs

- Complex sequencing & micro-operation logic
- Difficult to design and test
- Inflexible design
- Difficult to add new instructions

To Tie Up

- Control unit must control state of instruction cycle
- At end of each subcycle (fetch, indirect, execute, interrupt), the control unit instructs timing generator to re-initialise timing to T_1
- Control unit also sets appropriate values to P & Q (the control signals) to define next subcycle
- In modern processor, the number of Boolean instructions is very large (solution is to called *microprogramming)*

Outline

- Control Unit
  - Micro-operations
    - Fetch, Indirect, Interrupt and Execute cycles
    - Instruction cycle
  - Control of the Processor
    - Functional requirements and control signals
- Hardwired Implementation
- Micro-programmed Control
  - Basic concepts
    - Horizontal/Vertical microprogramming and control memory
  - Microinstruction Sequencing
    - Design and sequencing
    - Addressing
  - Microinstruction Execution
    - Taxonomy and encoding

Micro-programmed Control

- Use sequences of instructions (see earlier notes) to control complex operations
- In microprogramming language, each line describes set of micro-operations = microinstruction
- Sequence of microinstruction = microprogram
- Microprogramming a.k.a. firmware (because microprogram is like middleware between hardware and software)

Implementation (1)

- For each micro-op, all the control unit does is generate a set of control signals
- Each control signal (control line) is on or off
- This on/off state can be represented by a binary digit for each control line
- Have a control word for each micro-operation (each bit represents one control line)
- Have a sequence of control words for each machine code instruction (sequence of micro-ops)

Implementation (2)

- Since the sequence of micro-ops are not fix, can add control words in memory, each has unique address
- Then, each control word can have address field to specify the next micro-instruction, depending on conditions
- Can also add few bits to indicate condition

Implementation (3)

- Today's large microprocessor
  - Many instructions and associated register-level hardware
  - Many control points to be manipulated
- This results in control memory that
  - Contains a large number of words
    - co-responding to the number of instructions to be executed
  - Has a wide word width
    - Due to the large number of control points to be manipulated

Micro-program Word Length

- Based on 3 factors
  - Maximum number of simultaneous micro-operations supported
  - The way control information is represented or encoded
  - The way in which the next micro-instruction address is specified

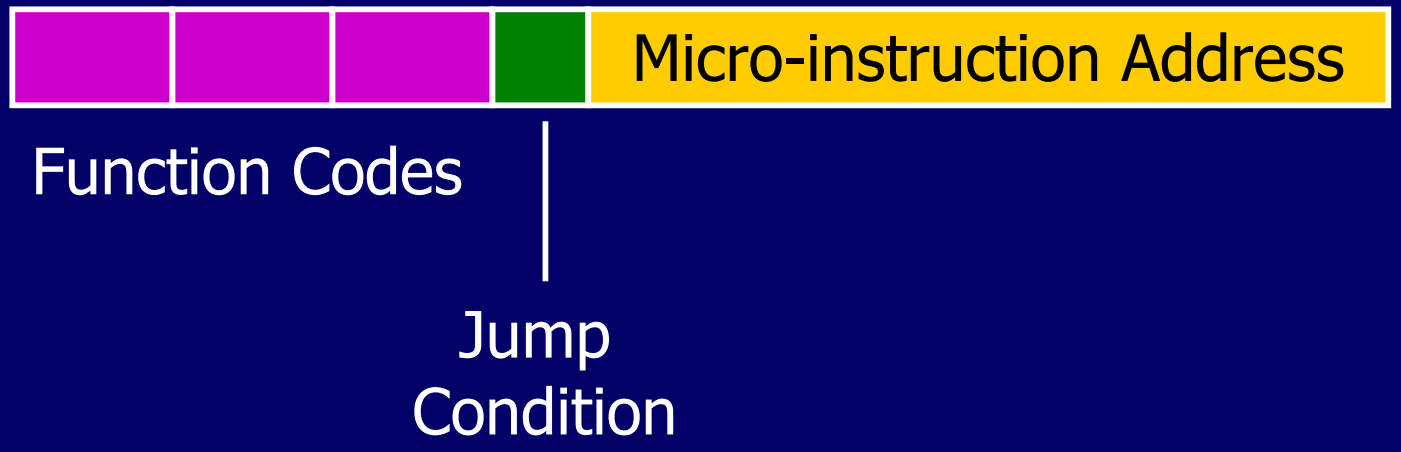
Micro-instruction Types

- Each micro-instruction specifies single (or few) micro-operations to be performed
  - (*vertical* micro-programming)
- Each micro-instruction specifies many different micro-operations to be performed in parallel
  - (*horizontal* micro-programming)

Vertical Micro-programming (1)

- Width is narrow (compact)
- n control signals encoded into $\log_2 n$ bits
- Limited ability to express parallelism
- Considerable encoding of control information requires external memory word decoder to identify the exact control line being manipulated

Vertical Micro-programming (2)

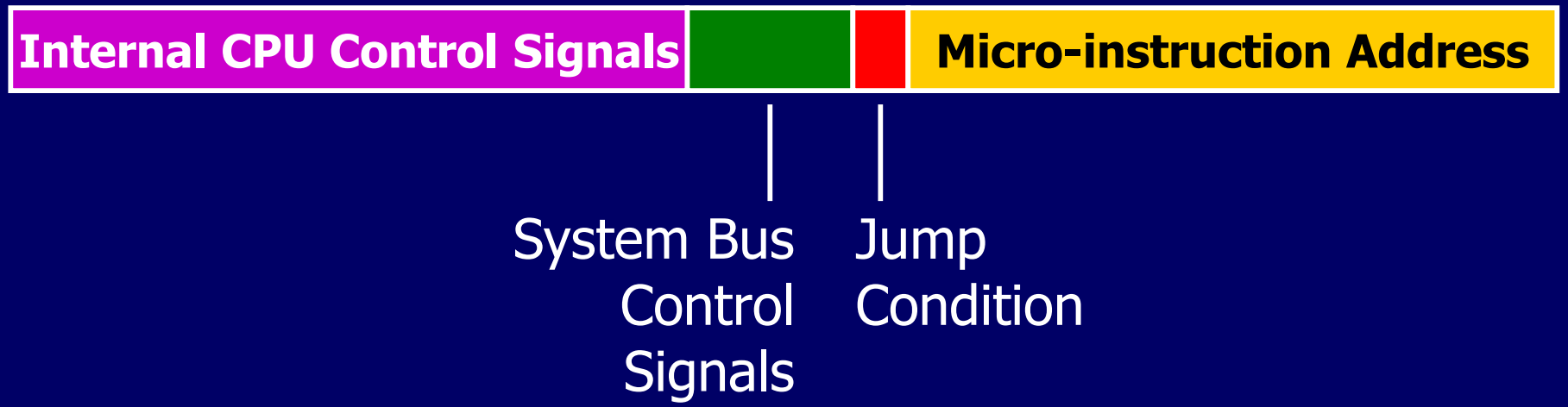


Function codes are decoded to issue control signal

Horizontal Micro-programming

- Wide memory word
- High degree of parallel operations possible
- Little encoding of control information

Horizontal Micro-programmed diag



Every bit in control field is attached to control line
(no decoding needed)

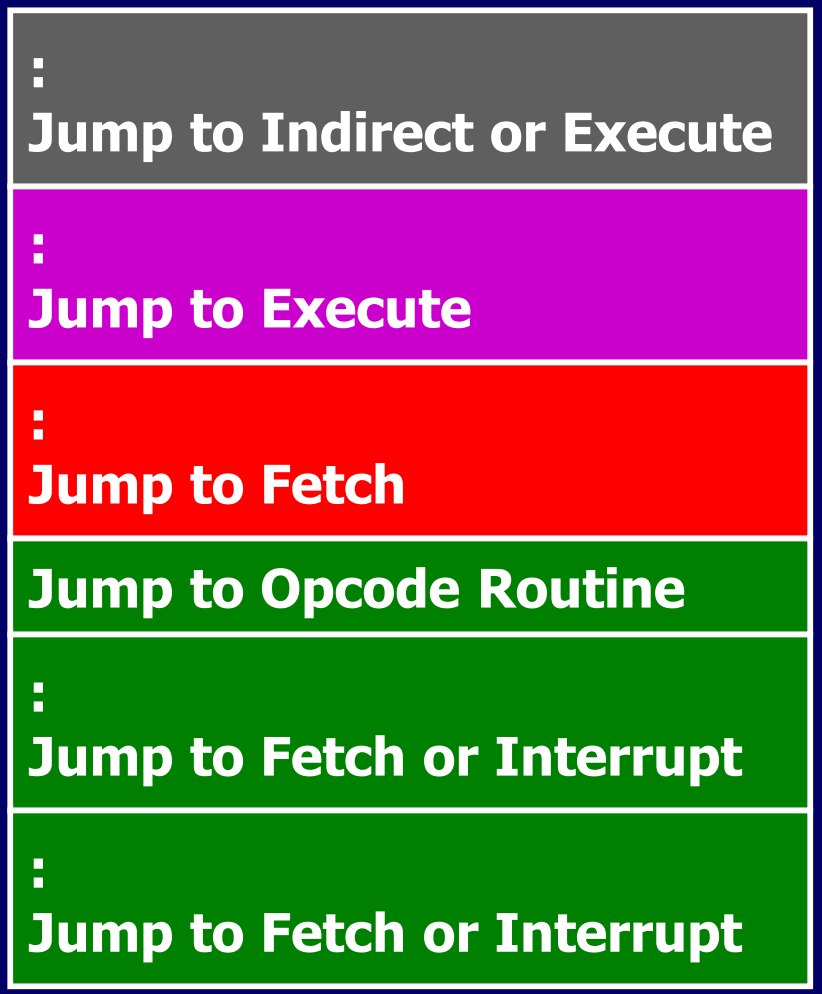
Compromise

- Divide control signals into disjoint groups
- Implement each group as separate field in memory word
- Supports reasonable levels of parallelism without too much complexity

Control Memory

- In control memory, microinstructions are executed sequentially
- Each routine ends with branch indicating where to go next

Control Memory



Control Unit Function (1)

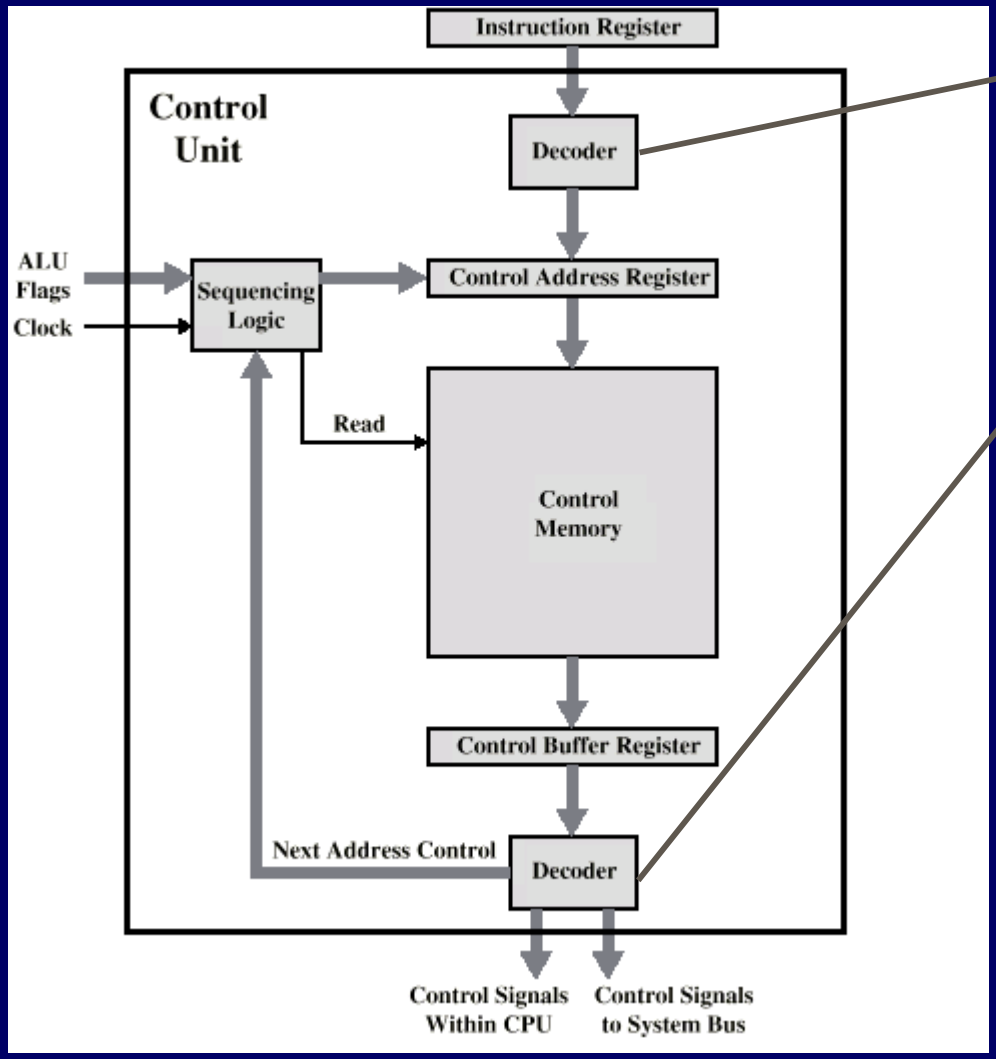
- Control memory contains a program describing the behaviour of the control unit (a set of micro instructions)
  - Sequence logic unit issues READ command (executing the microinstruction)
  - Word specified in control address register (has address of next microinstruction) is read into control buffer register
  - Control buffer register contents generates control signals (to the control lines) and next address information (for the sequencing logic unit)
  - Sequence logic loads new address into control address register based on next address information from control buffer register and ALU flags
- 4 steps = 1 clock pulse

Like the PC

Control Unit Function (2)

- At end of each microinstruction, sequencing logic unit loads new address into control address register.
- Depending on value of ALU flags and control buffer register, one of 3 decisions is made:
  - Next μ-instruction: Get next instruction: add 1 to control address register
  - Next instruction cycle: Jump to new routine (jump microinstruction): load address field of control buffer register into control address register
  - Next instruction: Jump to machine instruction routine: load control address register based on opcode in IR

Microprogrammed Control Unit



Decoder translates opcode of IR into control memory address

Used for vertical microinstructions (a code is used for each action and decoder translates code into individual control signals)
Pro: compact (fewer bits)
Con: additional logic required and time delay

Advantages and Disadvantages of Microprogramming

- Simplifies design of control unit
  - Cheaper
  - Less error-prone
  - (Hardwired control unit must contain complex logic whereas decoders and sequencing logic unit are simple pieces of logic)
- Slower (than hardwired)

- CISC usually use microprogramming, RISC prefer hardwire

Outline

- Control Unit
  - Micro-operations
    - Fetch, Indirect, Interrupt and Execute cycles
    - Instruction cycle
  - Control of the Processor
    - Functional requirements and control signals
- Hardwired Implementation
- Micro-programmed Control
  - Basic concepts
    - Horizontal/Vertical microprogramming and control memory
  - Microinstruction Sequencing
    - Design and sequencing
    - Addressing
  - Microinstruction Execution
    - Taxonomy and encoding

Tasks Done By Microprogrammed Control Unit

- 2 tasks of the microprogrammed control unit:
  - Microinstruction sequencing: get next microinstruction from control memory
  - Microinstruction execution: generate control signal
- Must consider both together

Microinstruction Sequencing: Design Considerations

- Size of microinstructions
  - Small control memory reduces cost
- Address generation time
  - Determined by instruction register (IR)
    - Once per cycle, after instruction is fetched
  - Next sequential address
    - Common in most designs
  - Branches
    - Both conditional and unconditional

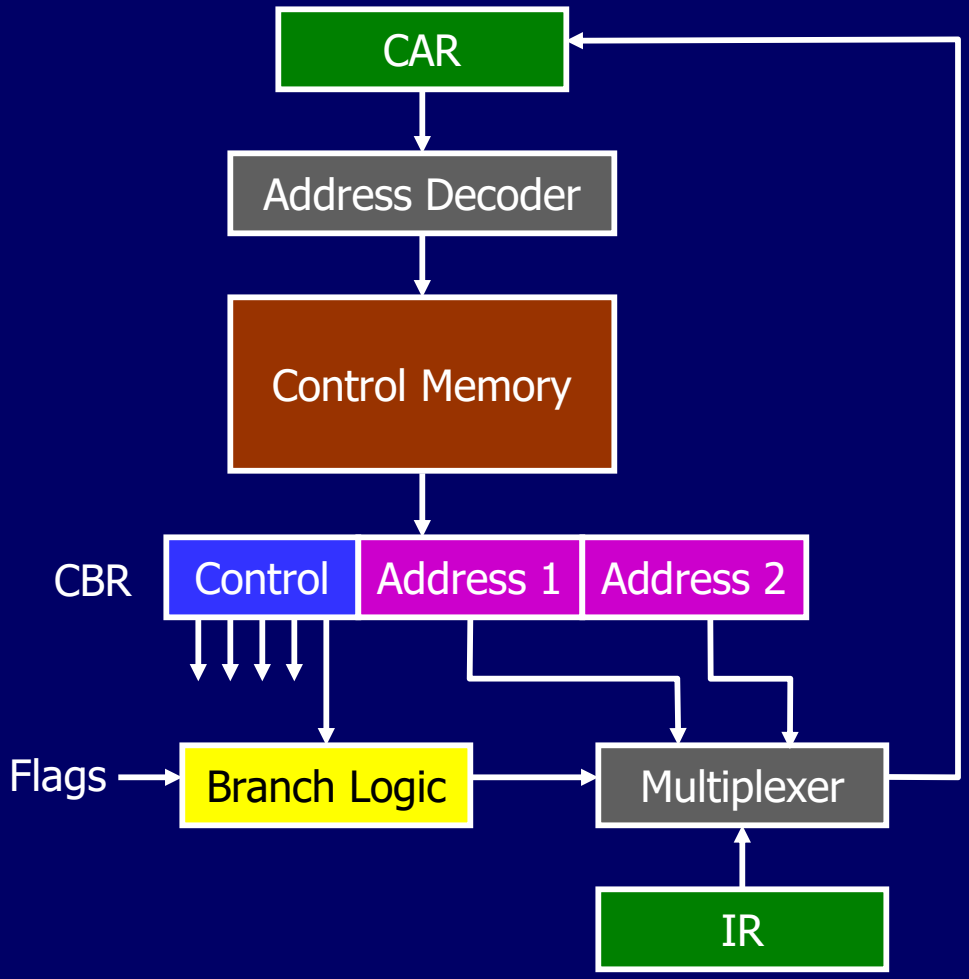
Sequencing Techniques

- Control memory address must be generated for next microinstruction
  - based on current microinstruction, condition flags, contents of IR
- 3 categories of techniques for address generation (based on format of address information):
  - Two address fields
  - Single address field
  - Variable format

2 Address Fields

- Multiplexer is destination for both address fields plus instruction register
- Based on address-selection input, multiplexer transmits either opcode or one of the 2 addresses to control address register (CAR)
- CAR is decoded to produce next microinstruction address
- Simple but requires more bits in microinstruction

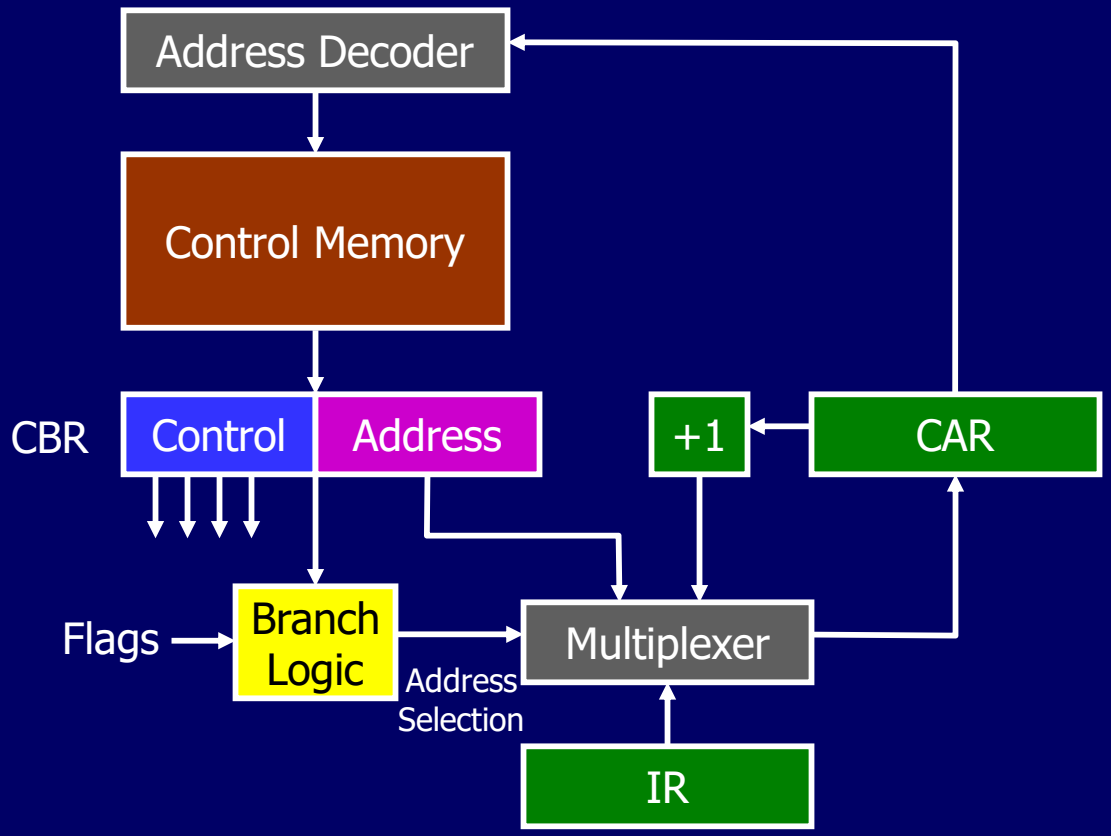
Branch Control Logic: 2 Address Fields



Single Address Fields

- Options for next address:
  - Address field
  - Instruction register code
  - Next sequential address
- Address-selection signals determine which option to select

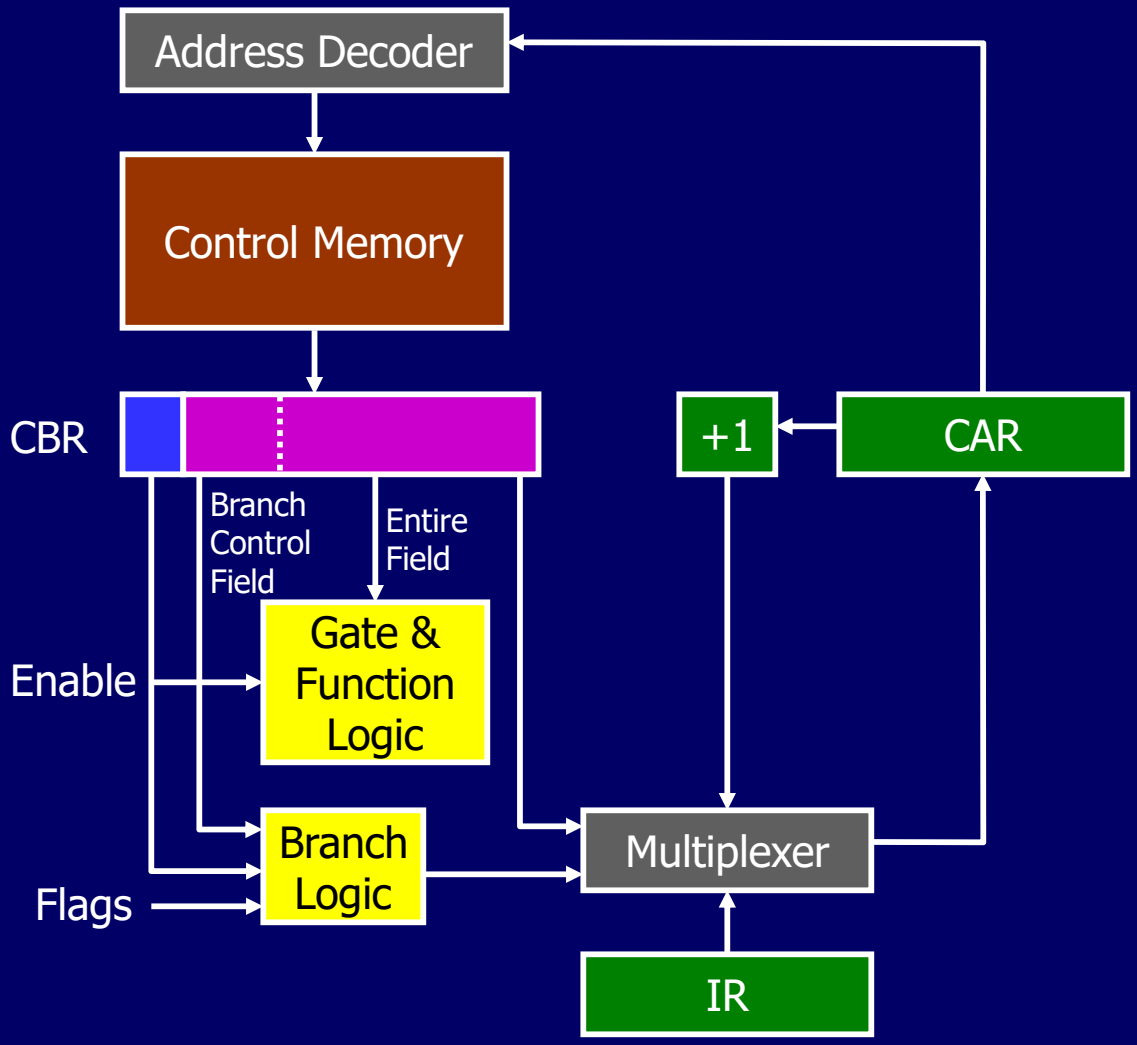
Branch Control Logic: Single Address Field



Variable Format

- 2 entirely different microinstruction formats
- One bit designates which format
- One format:
  - remaining bits used to activate control signals
  - next address is either next sequential address or derived from IR
- Other format:
  - some bits drive the branch logic module, remaining ones provide address
  - either conditional/unconditional branch is specified

Branch Control Logic: Variable Format



Address Generation (1)

- 2 techniques:
  - Explicit: address is explicitly available in the microinstruction
  - Implicit: additional logic is required to generate the address

Explicit	Implicit
Two-field	Mapping
Unconditional Branch	Addition
Conditional branch	Residual control

Address Generation (2)

- Explicit techniques:
  - Two-field: 2 alternative addresses are available with each microinstruction
  - Unconditional branch: using single address field or variable format
  - Conditional branch: depends on
    - ALU flags
    - part of opcode or address mode fields of machine instruction
    - parts of a selected register (e.g. sign bit)
    - status bits within the control unit

Address Generation (3)

- Implicit techniques:
  - Mapping
    - virtually all designs require this technique
    - opcode is mapped into microinstruction address
  - Adding (Combining)
    - taking 2 portions of an address to form the complete address
  - Residual control
    - involves using a microinstruction address that has been saved previously in temporary storage within the control unit
    - an internal register or stack of registers is used to hold return addresses

Outline

- Control Unit
  - Micro-operations
    - Fetch, Indirect, Interrupt and Execute cycles
    - Instruction cycle
  - Control of the Processor
    - Functional requirements and control signals
- Hardwired Implementation
- Micro-programmed Control
  - Basic concepts
    - Horizontal/Vertical microprogramming and control memory
  - Microinstruction Sequencing
    - Design and sequencing
    - Addressing
  - Microinstruction Execution
    - Taxonomy and encoding

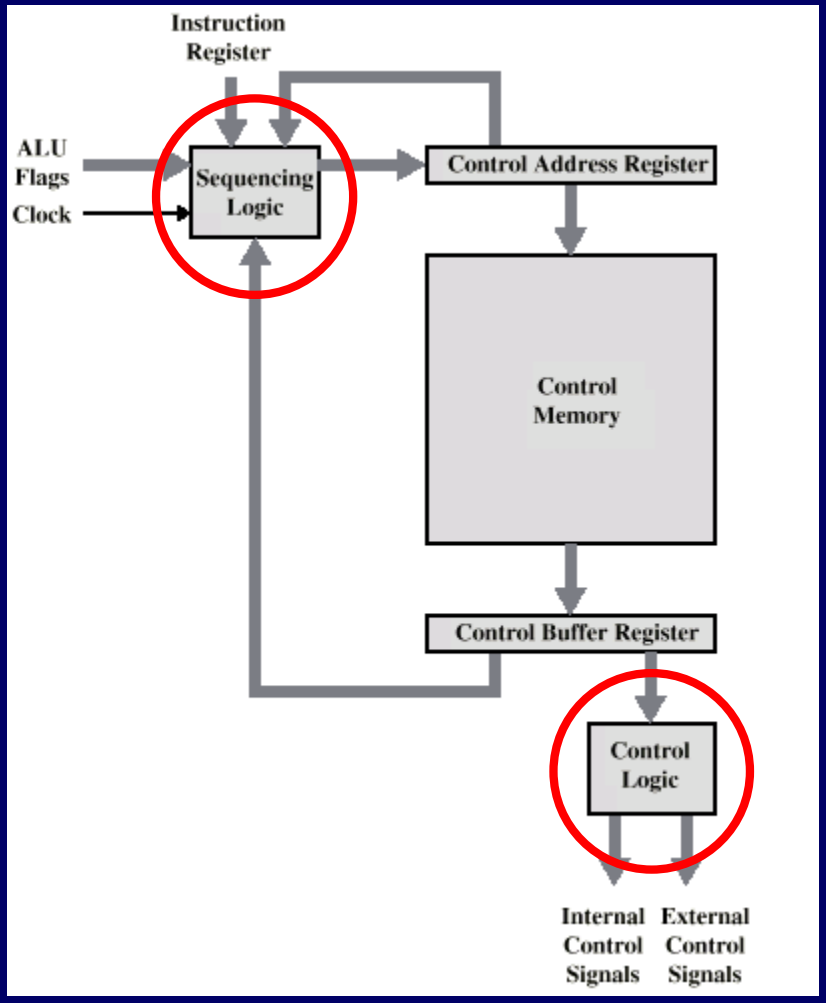
Microinstruction Execution

- The cycle is the basic event
- Each cycle is made up of two events
  - Fetch
    - Determined by generation of microinstruction address
  - Execute

Execute

- Effect is to generate control signals
- Some control points that are internal to processor
- Rest go to external control bus or other interface

Control Unit Organization (1)



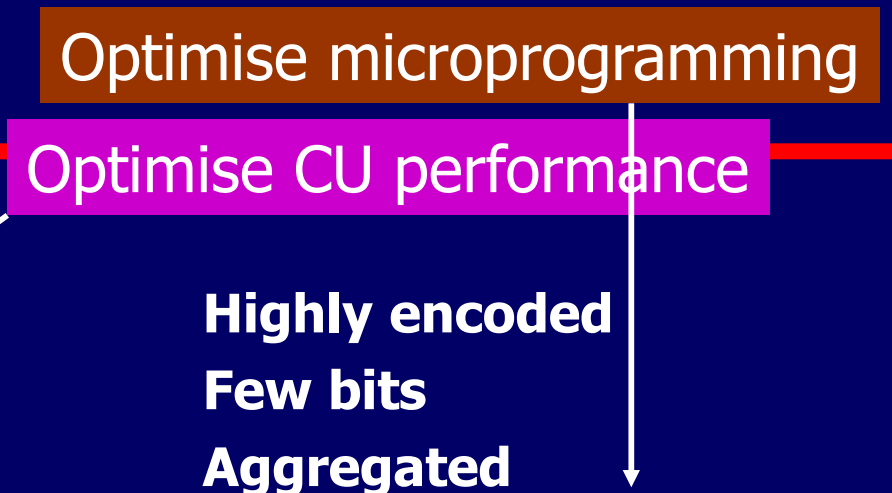
Control Unit Organization (2)

- Sequencing logic module contains logic to perform various functions
  - generates address of next microinstruction
    - using inputs from IR, ALU flags, Control Address Register and Control Buffer Register
- Control logic module generates control signals
  - as function of some bits in microinstruction

Microinstruction Taxonomy

- Types of classification:
  - Vertical/Horizontal
  - Packed/Unpacked
  - Hard/Soft microprogramming
  - Direct/Indirect encoding

Microinstruction Control Signal Spectrum (1)



- Characteristics:
  - **Unencoded** — **Highly encoded**
  - **Many bits** — **Few bits**
  - **Hardware view: Detail** — **Aggregated**
  - **Difficult to program** — **Easy to program**
  - **Concurrency: Fully exploit** — **Not fully exploit**
  - **Little or no control logic** — **Complex control logic**
  - **Fast execution** — **Slow execution**
  - **Optimise performance** — **Optimise programming**
- Terminology:
  - **Unpacked** — **Packed**
  - **Horizontal** — **Vertical**
  - **Hard** — **Soft**

Microinstruction Control Signal Spectrum (2)

- Degree of packing:
  - more packed, bits contain more information, more encoding
- Horizontal vs Vertical
  - relate to relative width of microinstructions
  - vertical = 16 to 40 bits
  - horizontal = 40 to 100 bits
- Hard vs Soft
  - degree of closeness to control signals and hardware
  - hard = fixed and committed to ROM
  - soft = more changeable and user microprogrammable

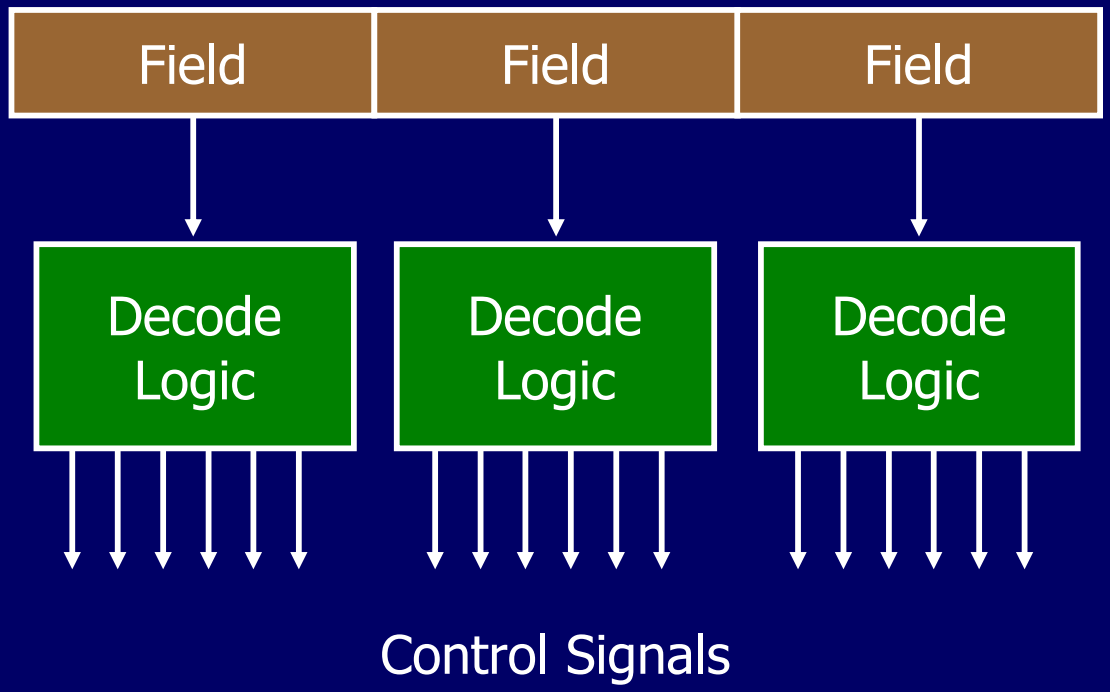
Microinstruction Encoding (1)

- Microprogrammed control units are not pure unencoded or horizontal (many bits) - some encoding is used
- Basic encoding:
  - microinstruction organised as set of fields
  - each field contains a code, after decoding it activates one or more control signals
- 2 approaches to organise encoded microinstruction into fields:
  - functional encoding
  - resource encoding

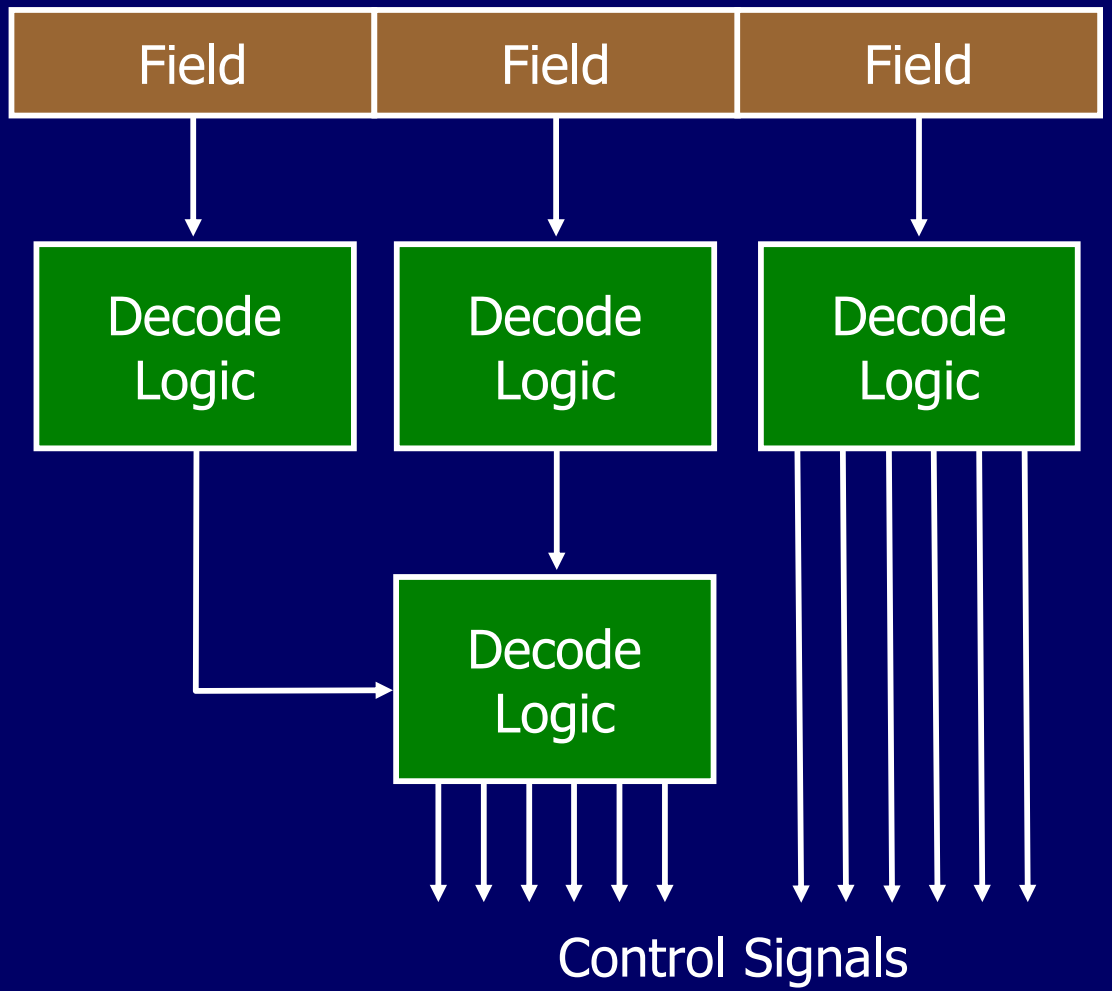
Microinstruction Encoding (2)

- Functional encoding identifies functions within the machine and designates fields by function type
- Resource encoding views machine as consisting of a set of independent resources and devotes one field to each
- Another aspect: direct or indirect encoding
  - indirect: one field is used to determine the interpretation of another field

Microinstruction Encoding (3) Direct Encoding

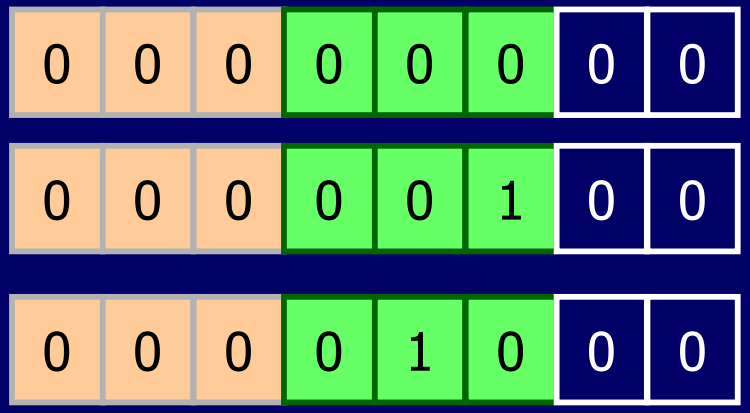


Microinstruction Encoding (4)
Indirect Encoding

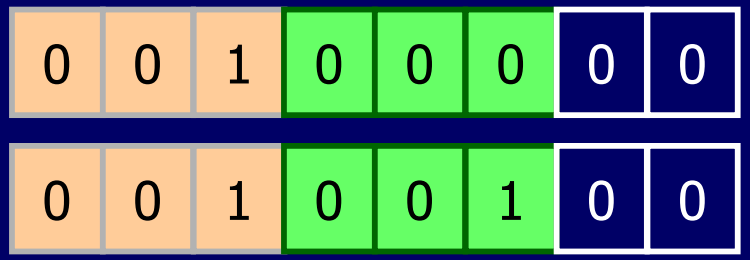


Microinstruction Encoding (5) Vertical microinstruction

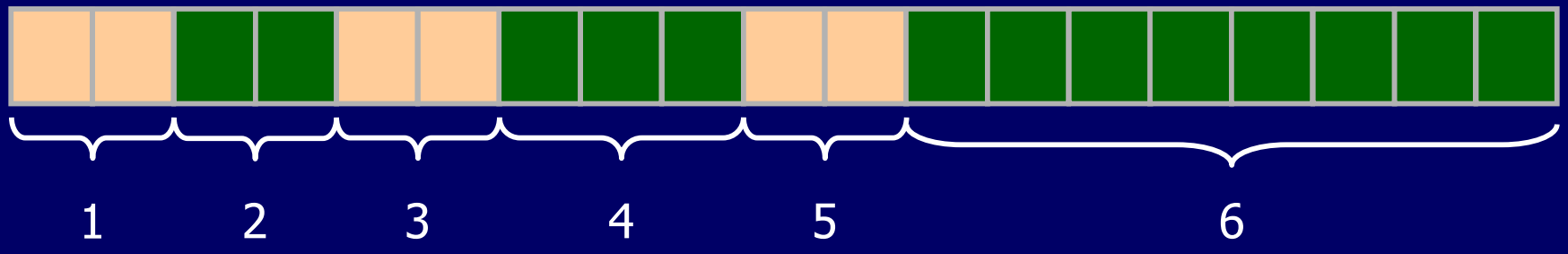
Register Transfers



Memory Operations



Microinstruction Encoding (6) Horizontal microinstruction



1 – Register Transfer
2 – Memory Operation
3 – Sequencing Operation
4 – ALU Operation
5 – Register Selection
6 – Constant